

Swagger/OpenApi

PASSO SEGUINTE: GERAR O SWAGGER / OPENAPI (CONTRACT-FIRST)

Isso **fecha definitivamente o planejamento técnico** e deixa o projeto em padrão profissional de engenharia.

Depois disso:

- a API fica 100% documentada
- a banca entende tudo sem você explicar
- o backend pode ser gerado quase automaticamente
- o frontend consome a API sem ambiguidade
- você pode até gerar clientes automaticamente no futuro

OPENAPI 3.0 — NEXUS ARTIFEX (API v1)

Você pode:

- colocar isso em `openapi.yaml`, ou
- integrar direto com Springdoc depois.

Base

```
openapi: 3.0.3
info:
  title: Nexus Artifex API
  description: Creative Evolution Engine API
  version: 1.0.0
servers:
  - url: /api/v1
```

PROJETOS

```
paths:
  /projects:
    post:
      summary: Criar novo projeto
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateProjectRequest'
      responses:
        '201':
          description: Projeto criado
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ProjectResponse'

    get:
      summary: Listar projetos
      responses:
        '200':
          description: Lista de projetos
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/ProjectResponse'
```

NÓS

```
/projects/{projectId}/nodes:
  post:
    summary: Criar nó inicial
```

```
parameters:
  - name: projectId
    in: path
    required: true
    schema:
      type: string
      format: uuid
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CreateNodeRequest'
responses:
  '201':
    description: Nó criado
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/NodeResponse'
```

```
/nodes/{nodeId}:
get:
  summary: Buscar nó por ID
  parameters:
    - name: nodeId
      in: path
      required: true
      schema:
        type: string
        format: uuid
  responses:
    '200':
      description: Nó encontrado
      content:
        application/json:
          schema:
```

\$ref: '#/components/schemas/NodeResponse'

EVOLUÇÃO CRIATIVA (SCAMPER)

```
/nodes/{nodeId}/evolve:
  post:
    summary: Evoluir um nó usando SCAMPER
    parameters:
      - name: nodeId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EvolutionRequest'
    responses:
      '200':
        description: Nós gerados pela IA
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/EvolutionResponse'
```

GRAFO

```
/projects/{projectId}/graph:
  get:
    summary: Obter grafo completo do projeto
    parameters:
```

```
- name: projectId
  in: path
  required: true
  schema:
    type: string
    format: uuid
responses:
  '200':
    description: Grafo do projeto
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/GraphResponse'
```

STREAM (SSE)

```
/projects/{projectId}/stream:
  get:
    summary: Stream de novos nós em tempo real
    parameters:
      - name: projectId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      '200':
        description: Server-Sent Events stream
        content:
          text/event-stream:
            schema:
              type: string
```

SCHEMAS (DTOs)

components:

schemas:

CreateProjectRequest:

type: object

properties:

name:

type: string

ProjectResponse:

type: object

properties:

id:

type: string

format: uuid

name:

type: string

createdAt:

type: string

format: date-time

CreateNodeRequest:

type: object

properties:

type:

type: string

enum: [CONCEPT, MUTATION]

label:

type: string

summary:

type: string

NodeResponse:

type: object

properties:

id:

type: string

format: uuid

label:

type: string

summary:

type: string

visualData:

type: object

additionalProperties: true

EvolutionRequest:

type: object

properties:

technique:

type: string

enum:

- SUBSTITUTE
- COMBINE
- ADAPT
- MODIFY
- PUT_TO_ANOTHER_USE
- ELIMINATE
- REVERSE

count:

type: integer

EvolutionResponse:

type: object

properties:

originalNodeid:

```
  type: string
  format: uuid
  technique:
    type: string
  generatedNodes:
    type: array
    items:
      $ref: '#/components/schemas/NodeResponse'
```

```
GraphResponse:
  type: object
  properties:
    nodes:
      type: array
      items:
        $ref: '#/components/schemas/NodeResponse'
    edges:
      type: array
      items:
        $ref: '#/components/schemas/EdgeResponse'
```

```
EdgeResponse:
  type: object
  properties:
    source:
      type: string
      format: uuid
    target:
      type: string
      format: uuid
    relationship:
      type: string
```

CHECKLIST ATUALIZADO (PLANEJAMENTO)

Item	Status
Arquitetura	
Diagramas	
Modelo de dados	
Contrato REST	
Swagger/OpenAPI	
Escopo definido	

 **Planejamento agora está 100% fechado.**